

INTERVIEW PROCESS & CHALLENGES

This process consists of **three components**:

- **A. Take-Home Coding Challenge** (completed remotely)
 - **B. Scoping & System Design Discussion – ChatRAG Tool** (in-person/online)
 - **C. GenAI Exercise** (completed remotely, discussed in-person/online)
-

A. TAKE-HOME CODING CHALLENGE

(Completed remotely, before the in-person interview)

You will receive a Coding Challenge (text + appendix) to complete on your own time.

Guidelines:

- You may take **as much time as you need**.
- The interview process continues **once the challenge has been submitted**.
- You are welcome to **ask questions** at any point.
- You may use helper tools (e.g., Google, ChatGPT) – please be **transparent** about doing so.
- You may ask for help, but transparency is expected.
- If you make assumptions, please **document them clearly**.

If you have any questions, please let us know.

PROCESS FOLLOWING CODING CHALLENGE REVIEW & APPROVAL

Please note that you will move forward to the in-person panel interview only if your take-home exercise meets our expectations.

Once it is reviewed and approved by the team, we will schedule a panel interview with the Solutions Engineering team, during which you will be asked to complete:

- **Challenge B: Scoping & System Design** (30-45 minutes)
- **Challenge C: GenAI Exercise** (30-45 minutes)

Challenge C: Please share the deliverables with us before the panel interview.

Each exercise is timed and will be conducted **in person or online**.

Best regards
Squirro

Delivery Coding Challenge A:

Use the New York Times API (specifically Article Search) to return news items in a Python dictionary format. See developer.nytimes.com for the API documentation.

You must adhere to the template in the appendix below, specifically `getDataBatch`` should return the results in batches. You can verify the result by running that file with Python 3.

Each result should contain the news results from the NYTimes API as a flattened dictionary. The flattened dictionary must contain all elements for each retrieved document. Example for such a dictionary (excerpt):

```
{
    "web_url": "http://nytimes.com/...",
    "headline.main": "The main headline",
    "headline.kicker": "...",
    ...
}
```

Don't use a third-party library for dictionary flattening; instead, define a function for it as part of the solution.

Bonus points for a solution that loads the data incrementally.

Another bonus point would be for the solution to return a dynamic schema. That is, the `getSchema` function returns the schema of the flattened dictionary.

Please also provide a `requirements.txt` file in case your solution depends on a third-party library and a `README` file with any relevant information.

Background

The template file is a stripped-down version of the Squirro data loader plugins. This is what we use at Squirro to connect to a new data source. While not required for this task, you may want to look at the data loader documentation.

Data Loading in Squirro:

<https://docs.squirro.com/en/latest/technical/data-loading/index.html>

Data Loader Plugins:

<https://docs.squirro.com/en/latest/technical/data-loading/plugins/index.html>

Appendix

```
import argparse
import logging

"""
Skeleton for Squirro Delivery Hiring Coding Challenge
November 2025
"""

log = logging.getLogger(__name__)

class NYTimesSource(object):
    """
    A data loader plugin for the NY Times API.
    """

    def __init__(self):
        pass

    def connect(self, inc_column=None, max_inc_value=None):
        """Connect to the source"""
        log.debug("Incremental Column: %r", inc_column)
        log.debug("Incremental Last Value: %r", max_inc_value)

    def disconnect(self):
        """Disconnect from the source."""
        # Nothing to do
        pass

    def getDataBatch(self, batch_size):
        """
        Generator - Get data from source on batches.

        :returns One list for each batch. Each of those is a
list of dictionaries with the defined rows.
```

```

        """
        # TODO: implement - this dummy implementation returns
one batch of data
        yield [
            {
                "headline.main": "The main headline",
                "_id": "1234",
            }
        ]

    def getSchema(self):
        """
        Return the schema of the dataset
        :returns a List containing the names of the columns
retrieved from the
        source
        """

        schema = [
            "title",
            "body",
            "created_at",
            "id",
            "summary",
            "abstract",
            "keywords",
        ]

        return schema

if __name__ == "__main__":
    config = {
        "api_key": "NYTIMES_API_KEY",
        "query": "Silicon Valley",
    }
    source = NYTimesSource()

    # This looks like an argparse dependency - but the Namespace
class is just
    # a simple way to create an object holding attributes.
    source.args = argparse.Namespace(**config)

    for idx, batch in enumerate(source.getDataBatch(10)):
        print(f"{idx} Batch of {len(batch)} items")
        for item in batch:

```

```
print(f"    - {item['_id']} -  
{item['headline.main']}")
```

After your Coding Challenge has been submitted:

You will be invited to a panel interview, where you will complete Challenge B (System Design) and present Challenge C (GenAI Exercise) in person or online. Each exercise will be individually timed and limited to approximately 30 minutes.

B. SCOPING & SYSTEM DESIGN DISCUSSION – RAG chatbot (30 minutes)

Objective

We want to see how you **scope an ambiguous project**, ask clarifying questions, and design at a functional level.

You do **not** need to draw a diagram, just walk us through your thought process. We are evaluating your clarity of thought and ability to reason about components.

Candidate Brief

You are meeting with a client who wants to build an internal **RAG chatbot**. The goal of the tool:

- Allow employees to ask questions in a user interface.
- Utilize an LLM to help answer questions based on information found in **internal** documents (instead of the general LLM knowledge base).

Your task during the interview:

1. **Ask scoping questions**
Anything needed to design the system. Goals, constraints, user groups, data types, expected outputs, etc.
2. **Design the high-level flow of the system**
Walk us through how you would structure:

- Data ingestion
 - Document preparation (chunking, embeddings)
 - Indexing
 - Retrieval at query time
 - LLM answer generation
 - How citations are returned to the UI
3. **Explain your choices**
- Why you would choose that architecture
 - What assumptions you are making
 - Where you might add fallbacks or improvements

No technical depth beyond this is required.
This is about **how you think**.

C. GenAI Exercise – 30 minutes

Please complete the following exercise before our panel interview:

1. Spin up a Squirro Instance

- Go to <https://start.squirro.com/setup/app>
- Create a new project using the “**Squirro Chat – Company Data**” template.

2. Load Your Own Data

- Prepare a small dataset (for example: upload a **ZIP file containing several PDF documents**).
- Add this data to the project so the chat can use it.
- Make sure the **chat is working with your uploaded content**.

3. Create a Custom Tool

- Follow the documentation here:
<https://docs.squirro.com/en/latest/technical/agents/developing-custom-tools.html>
- You will receive the **squirro_toolbox** package as part of the exercise materials.
- Using it, develop a simple custom tool of your choice.
- Upload the tool to your Squirro instance.

Important:

Your custom tool **will not appear** in the UI yet — this is expected. Activating it requires restarting an internal Squirro service, which **only works via SSH**, and you will *not* have SSH access.

For this task, it is sufficient that you **successfully upload the tool**.

4. Deliverables

Please send us:

1. The **URL of your Squirro instance** where the tool was uploaded.
2. Any short notes you want to share about your approach (optional).
3. Go to <https://start.squirro.com/admin/instances> and invite as an admin by email the people who interviewed you.

We will review and discuss the tool implementation together during the interview.